

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	K ABISHEK	Reg. No.:	192521120
Section / Batch:	IT	Date:	28/07/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I K ABISHEK/192521120(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K ABISHEK

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Binary Search – Search for Highest Existing Element

Problem Statement

Given the sorted dataset:

[8, 16, 24, 32, 40, 48, 56, 64]

the objective is to implement the **Binary Search algorithm** to find the target element **64**. The program should search the ordered dataset, display the steps involved in reducing the search interval, count the total number of comparisons made, and analyze the efficiency of the algorithm.

Since the dataset is already sorted in ascending order, Binary Search is suitable because it repeatedly divides the search interval into two halves. Instead of checking every element sequentially, it compares the target with the middle element and eliminates the half that cannot contain the target.

Algorithm Implementation

Source Code

```
def binary_search(arr, target):  
    low = 0  
    high = len(arr) - 1  
    comparisons = 0  
  
    while low <= high:  
        mid = (low + high) // 2  
        comparisons += 1  
  
        print("Comparison", comparisons,  
              ": low =", low,  
              ", high =", high,  
              ", mid =", mid,
```

```
    ", element =", arr[mid])
```

```
    if arr[mid] == target:
```

```
        return mid, comparisons
```

```
    elif arr[mid] < target:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
    return -1, comparisons
```

```
# Sorted dataset
```

```
arr = [8, 16, 24, 32, 40, 48, 56, 64]
```

```
# Target element
```

```
target = 64
```

```
# Perform Binary Search
```

```
position, comparisons = binary_search(arr, target)
```

```
# Display result
```

```
if position != -1:
```

```
    print("\nTarget element", target, "found at index", position)
```

```
else:
```

```
    print("\nTarget element not found")
```

```
print("Total number of comparisons:", comparisons)
```

Sample Output

Comparison 1 : low = 0 , high = 7 , mid = 3 , element = 32

Comparison 2 : low = 4 , high = 7 , mid = 5 , element = 48

Comparison 3 : low = 6 , high = 7 , mid = 6 , element = 56

Comparison 4 : low = 7 , high = 7 , mid = 7 , element = 64

Target element 64 found at index 7

Total number of comparisons: 4

Step-by-Step Execution

The dataset contains 8 sorted elements:

[8, 16, 24, 32, 40, 48, 56, 64]

Target element = **64**

Comparison 1

Initial search interval:

low = 0, high = 7

Calculate middle position:

mid = (0 + 7) // 2 = 3

Element at index 3:

arr[3] = 32

Comparison:

64 > 32

Therefore, the target must be in the **right half**.

New search interval:

low = 4, high = 7

Remaining elements:

[40, 48, 56, 64]

Comparison 2

Current search interval:

low = 4, high = 7

Calculate middle position:

mid = (4 + 7) // 2 = 5

Element at index 5:

arr[5] = 48

Comparison:

64 > 48

Therefore, discard the left portion of the current interval.

New search interval:

low = 6, high = 7

Remaining elements:

[56, 64]

Comparison 3

Current search interval:

low = 6, high = 7

Calculate middle position:

mid = (6 + 7) // 2 = 6

Element at index 6:

arr[6] = 56

Comparison:

64 > 56

Therefore, the target must be to the right.

New search interval:

low = 7, high = 7

Remaining element:

[64]

Comparison 4

Current search interval:

low = 7, high = 7

Calculate middle position:

mid = (7 + 7) // 2 = 7

Element at index 7:

arr[7] = 64

Comparison:

64 == 64

Therefore, the target element is found successfully.

Position = Index 7

Total Comparisons = 4

Search Interval Reduction

Comparison Low High Mid Middle Element Decision

1	0	7	3	32	$64 > 32 \rightarrow$ Search right
2	4	7	5	48	$64 > 48 \rightarrow$ Search right
3	6	7	6	56	$64 > 56 \rightarrow$ Search right
4	7	7	7	64	$64 = 64 \rightarrow$ Found

The search interval is reduced as follows:

8 elements → 4 elements → 2 elements → 1 element

This demonstrates the main working principle of Binary Search: the search space is approximately halved after every unsuccessful comparison.

Comparison Count

The target element **64** is the largest element in the dataset. Therefore, it is located at the final position.

The algorithm performs the following comparisons:

1. **64 with 32**
2. **64 with 48**
3. **64 with 56**
4. **64 with 64**

Therefore:

Total number of comparisons = 4

The target is successfully found at **index 7**.

Time Complexity Analysis

Binary Search eliminates approximately half of the remaining elements after each comparison.

For an input size of **n**, the search interval reduces as:

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$

The number of divisions required is approximately:

$\log_2 n$

Therefore, the time complexity is:

Best Case = $O(1)$

The best case occurs when the target is found at the first middle position.

Average Case = $O(\log n)$

On average, the search requires logarithmic time because the search interval is repeatedly divided into halves.

Worst Case = $O(\log n)$

In the worst case, the algorithm continues dividing the search interval until only one element remains.

For this dataset:

$$n = 8$$

$$\log_2 8 = 3$$

The target 64 requires **4 actual element comparisons** because the final comparison is needed to verify the remaining element.

Thus, the implementation performs **4 comparisons** for this particular search.

Space Complexity

The implementation uses the variables low, high, mid, and comparisons.

No additional data structure proportional to the input size is created.

Therefore:

$$\text{Auxiliary Space Complexity} = O(1)$$

The iterative implementation uses constant extra memory.

Correctness Verification

The algorithm is correct because the input dataset is sorted in ascending order.

At every step:

- If $\text{arr}[\text{mid}] == \text{target}$, the target is found.
- If $\text{arr}[\text{mid}] < \text{target}$, all elements at or before mid can be eliminated because they are smaller than the target.
- If $\text{arr}[\text{mid}] > \text{target}$, all elements at or after mid can be eliminated because they are greater than the target.

For the given input:

$$64 > 32 \rightarrow \text{right half}$$

$$64 > 48 \rightarrow \text{right half}$$

64 > 56 → right half

64 = 64 → target found

Hence, the algorithm correctly identifies **64 at index 7**.

Efficiency When the Target Is the Largest Element

In this example, the target **64** is the largest element in the dataset. Therefore, Binary Search repeatedly moves toward the right side of the array.

Although the target is at the last position, the algorithm does not check all 8 elements individually. It eliminates approximately half of the remaining search space after each comparison.

The search is completed in only **4 comparisons**, compared with **8 comparisons** that may be required by a linear search in the worst case.

Therefore, Binary Search is more efficient than Linear Search for large **sorted datasets**, with a time complexity of **$O(\log n)$** compared with **$O(n)$** for Linear Search.

Final Result

Input Dataset: [8, 16, 24, 32, 40, 48, 56, 64]

Target: 64

Result: Element found successfully

Index: 7

Total Comparisons: 4

Time Complexity: $O(\log n)$

Auxiliary Space Complexity: $O(1)$

The implementation successfully finds the highest existing element, **64**, using Binary Search. The search interval is reduced from 8 elements to 4, then 2, and finally 1 element. This demonstrates that Binary Search is an efficient searching technique for sorted datasets, even when the target is the largest element.

OUTPUT

Comparison 1 : low = 0 , high = 7 , mid = 3 , element = 32

Comparison 2 : low = 4 , high = 7 , mid = 5 , element = 48

Comparison 3 : low = 6 , high = 7 , mid = 6 , element = 56

Comparison 4 : low = 7 , high = 7 , mid = 7 , element = 64

Target element 64 found at index 7

Total number of comparisons: 4